MODULE 05 | PROCESSES

Processes

See what is running, stop what is stuck, and run work in the background.

Mission	Topic	Commands / Concepts
1	List Processes: ps, pgrep and pstree	ps, ps aux, pgrep, pstree
2	Monitor Live: top, htop, free, uptime	top, htop, free, uptime, vmstat
3	Stop Processes: kill and Signals	kill, killall, pkill, signals
4	Job Control: &, jobs, fg, bg	&, jobs, fg, bg, Ctrl+Z, wait
5	Priority and Persistence: nice, renice, nohup	nice, renice, nohup, disown, screen/tmux
Boss	Runaway Process	Everything in this module

How to use this guide: read each mission, type every example yourself in the Tux Tutor terminal, attempt the practice tasks, then check the answers at the end. Use a throwaway practice folder so mistakes cost nothing.

Module Overview

What is a process?

A **process** is a running program. Every process has a unique **PID** (process ID), a parent (**PPID**), an owner, a state and resource usage. The first process (`systemd` or `init`) has PID 1 and starts everything else.

State code	Meaning
`R`	Running or ready to run
`S`	Sleeping, waiting for an event (most processes)
`D`	Uninterruptible sleep (usually waiting on disk)
`T`	Stopped (for example Ctrl+Z)
`Z`	Zombie: finished, but parent has not collected it

Foreground vs background

- **Foreground**: owns the terminal; you wait until it ends.
- **Background**: runs while you keep using the terminal (add `&`).
- **Daemon / service**: background process with no terminal, often started at boot.

```
$ sleep 300 &            # a handy dummy process for practice
[1] 4821
```

MISSION 1 | +50 XP

List Processes: ps, pgrep and pstree

Goal: Take a snapshot of running processes and find the one you want.

```
$ ps                      # only your current shell's processes
  PID TTY          TIME CMD
 3012 pts/0    00:00:00 bash
 4900 pts/0    00:00:00 ps
$ ps aux                  # every process, user-oriented columns
$ ps -ef                  # every process, full format (shows PPID)
$ ps -u haani             # processes of one user
$ ps -p 4821              # one PID
$ ps -o pid,ppid,%cpu,%mem,cmd -p 4821   # choose your columns
```

ps aux column	Meaning
USER	Owner
PID	Process ID
%CPU %MEM	Share of CPU and memory
VSZ RSS	Virtual / physical memory (KB)
STAT	State code (R, S, Z...)
START TIME	Start time and total CPU time
COMMAND	Command line

Finding processes

```
$ ps aux | grep sleep             # also matches the grep itself
$ ps aux | grep [s]leep           # trick to hide the grep line
$ pgrep sleep                     # just the PIDs
$ pgrep -l ssh                    # PID and name
$ pgrep -u haani bash
$ ps aux --sort=-%mem | head -5   # top memory users
$ ps aux --sort=-%cpu | head -5   # top CPU users
```

Process tree

```
$ pstree -p | head
systemd(1)-+-sshd(842)---sshd(3001)---bash(3012)---pstree(4950)
$ ps -ef --forest
```

Practice

- Start `sleep 300 &`, then find its PID in three different ways.
- Show the five processes using the most memory.
- Find the parent PID of your current shell (`echo $$` gives its PID).

MISSION 2 | +50 XP

Monitor Live: top, htop, free, uptime

Goal: Watch CPU and memory in real time and read the load average.

```
$ top
```

Top header lines show uptime, **load average** (1, 5, 15 minute averages), task counts, CPU split and memory. The list below is sorted by CPU use.

Key in top	Action
`q`	Quit
`M`	Sort by memory
`P`	Sort by CPU
`k`	Kill a process (asks for PID)
`1`	Show each CPU core
`u`	Filter by user
`h`	Help

Reading load average

Load average is the number of processes wanting CPU. On a machine with **4 cores**, a load of 4.0 means fully busy; well above 4 means a queue is building.

```
$ uptime
 10:42:01 up 3 days, 2:11, 2 users, load average: 0.45, 0.60, 0.52
$ nproc            # number of CPU cores
```

Memory and more

```
$ free -h
              total   used   free  shared  buff/cache  available
Mem:          7.7Gi   2.1Gi  3.2Gi   0.2Gi       2.4Gi      5.1Gi
$ vmstat 2 5       # system stats every 2 s, 5 times
$ htop             # friendlier top (install: sudo apt install htop)
```

> **Tip**
> Judge free memory by the **available** column. Linux uses spare RAM as cache, so a low 'free' number is normal.

Practice

- Open `top`, sort by memory, then quit.
- Read `uptime` and `nproc`. Is your system busy?
- Show memory in human-readable units with `free`.

MISSION 3 | +50 XP

Stop Processes: kill and Signals

Goal: Ask processes to stop politely, and force them when necessary.

`kill` does not just kill. It **sends a signal** to a process.

Signal	Number	Meaning
`SIGTERM`	15	Please terminate (default). Process may clean up.
`SIGKILL`	9	Force stop immediately. Cannot be caught.
`SIGHUP`	1	Hangup; many daemons reload their config
`SIGINT`	2	Interrupt (what Ctrl+C sends)
`SIGSTOP`	19	Pause the process
`SIGCONT`	18	Resume a paused process

```
$ sleep 500 &
[1] 5120
$ kill 5120               # SIGTERM, polite
$ kill -9 5120            # SIGKILL, last resort
$ kill -SIGSTOP 5120      # pause
$ kill -SIGCONT 5120      # resume
$ kill -l                 # list all signals
```

By name

```
$ pkill sleep               # signal processes matching a name
$ pkill -f "python app.py"  # match the full command line
$ killall firefox           # every process with this exact name
$ pkill -u bob              # all of bob's processes (careful!)
```

Recommended order

- 1. `kill PID` (SIGTERM) and wait a few seconds.
- 2. `kill -HUP PID` if it is a service that can reload.
- 3. `kill -9 PID` only if it ignores SIGTERM. It gives no chance to save data or remove temp files.

Warning
You can only signal your own processes. Killing others needs `sudo`. Never kill PID 1 or random system processes.

Tip
Find, then kill: `pgrep -l sleep` first to confirm the target, then `pkill sleep`.

Practice

- Start three `sleep` processes and stop them all with one command.
- Pause a process with `SIGSTOP`, confirm state `T` in `ps`, and resume it.

- Why is `kill -9` considered a last resort?

MISSION 4 | +50 XP

Job Control: &, jobs, fg, bg

Goal: Move commands between foreground and background in your shell.

```
$ sleep 100 &                # start in background
[1] 5301
$ sleep 200                   # runs in foreground, terminal is busy
^Z                            # Ctrl+Z suspends it
[2]+  Stopped   sleep 200
$ jobs
[1]-  Running   sleep 100 &
[2]+  Stopped   sleep 200
$ bg %2                       # resume job 2 in the background
$ fg %1                       # bring job 1 to the foreground
^C                            # Ctrl+C ends it
```

Command	Meaning
`cmd &`	Start in background
`Ctrl+Z`	Suspend the foreground job
`jobs`	List this shell's jobs (`-l` adds PIDs)
`bg %n`	Resume job n in the background
`fg %n`	Bring job n to the foreground
`kill %n`	Signal job n
`wait`	Pause until background jobs finish

Useful example

```
$ ( sleep 5; echo done ) &
$ wait
done
```

Jobs vs processes
Job numbers like `%1` belong to **one shell**. PIDs are system-wide. Closing the terminal normally ends its jobs, which the next mission solves.

Practice

- Start a long `sleep`, suspend it, then resume it in the background.
- Run three background jobs and use `wait` to block until all finish.
- Bring a background job to the foreground and cancel it with Ctrl+C.

MISSION 5 | +50 XP

Priority and Persistence: nice, renice, nohup

Goal: Control how much CPU a process gets and keep it running after you log out.

Niceness

Niceness ranges from **-20** (highest priority) to **19** (lowest). Default is 0. A 'nice' process is polite and lets others go first.

```
$ nice -n 10 ./heavy_job.sh          # start with lower priority
$ sudo nice -n -5 ./important.sh     # higher priority needs root
$ renice 15 -p 5301                  # change a running process
$ ps -o pid,ni,cmd -p 5301           # see the NI column
$ sudo renice -n -5 -p 5301
```

nohup - survive logout

When a terminal closes, the shell sends **SIGHUP** to its jobs and they die. `nohup` makes a command ignore it.

```
$ nohup ./backup.sh > backup.log 2>&1 &
[1] 6001
$ tail -f backup.log
```

Without redirection, `nohup` writes output to `nohup.out`.

If it is already running

```
$ long_task                 # oops, forgot nohup
^Z
$ bg                         # move to background
$ disown -h %1              # detach so SIGHUP will not reach it
```

Better for interactive work

- `tmux` or `screen` keep a whole terminal session alive, even if SSH drops.
- For permanent services, write a systemd unit instead (see Module 08).

Practice

- Run `sleep 300` with niceness 15 and verify it with `ps -o ni`.
- Start a command with `nohup`, close the terminal, and check from a new one that it still runs.
- Increase the niceness of a running process with `renice`.

BOSS BATTLE | +150 XP

Boss: Runaway Process

Goal: combine everything from this module without hints.

Scenario

A CPU-hungry script is slowing down the server. You must find it, calm it down, keep a useful job alive, and clean up.

Tasks

1. Start a CPU hog: `yes > /dev/null &` and note its PID.
2. Find it with `pgrep` and `ps`, confirming high `%CPU`.
3. Lower its priority to niceness 15 with `renice`.
4. Start `sleep 600` under `nohup` in the background, writing to `sleep.log`.
5. List all your jobs, then bring the `yes` job to the foreground and stop it with Ctrl+C.
6. Politely terminate any remaining `yes` processes, then force-kill only if needed.
7. Show the five biggest memory users.
8. Confirm no `yes` process is left.

Sample solution

```
yes > /dev/null &
pgrep -l yes
ps aux | grep [y]es
renice 15 -p $(pgrep yes)
nohup sleep 600 > sleep.log 2>&1 &
jobs -l
fg %1        # then press Ctrl+C
pkill yes || true
ps aux --sort=-%mem | head -6
pgrep yes || echo "clean"
```

Check Your Understanding

Q1. What does PID stand for?

Q2. Difference between `ps` and `top`?

Q3. What does `kill 1234` send by default?

Q4. Which signal cannot be ignored?

Q5. How do you start a command in the background?

Q6. What does Ctrl+Z do?

Q7. What does `nohup` prevent?

Q8. Niceness range?

Q9. What is a zombie process?

Q10. Which command lists background jobs?

Answers

#	Answer
1	Process ID.
2	ps is a snapshot; top updates live.
3	SIGTERM (15).
4	SIGKILL (9).
5	Add & at the end.
6	Suspends the foreground job.
7	The process being killed by SIGHUP when the terminal closes.
8	-20 (highest priority) to 19 (lowest).
9	A finished process whose parent has not collected its exit status.
10	jobs

Quick Reference Cheat Sheet

Command	Purpose	Example
ps aux	All processes	ps aux | grep nginx
pgrep / pkill	Find / signal by name	pkill -f app.py
top / htop	Live monitor	top
free -h	Memory	free -h
uptime	Load average	uptime
kill / kill -9	Stop / force stop	kill -9 PID
jobs / fg / bg	Job control	fg %1

Command	Purpose	Example
cmd &	Run in background	sleep 9 &
nice / renice	Priority	renice 10 -p PID
nohup	Survive logout	nohup cmd &
pstree	Process tree	pstree -p

Up next: Module 06
Networking. Use ping, curl, ssh, ss and dig to test connectivity and talk to other machines.